Lab # 04

COURSE CODE: SE-206L

COURSE TITLE:

Software Construction & Development

LAB TITLE

Data Types, Mutability, and Collections

LAB ASSESSMENT TABLE:

Obtained Marks	Total Marks

Submitted To: Engr. Rizwan Shah.

Submitted By: Muhammad Danial.

Registration – No: 24ABSWE0021.

Semester: 5th

Date: 09-09-2026.

Dept. of Software Engineering

UET Peshawar, Abbottabad Campus.

Instructor Sign : _____

Lab 4:

Data Types, Mutability, and Collections

Repository Link: https://github.com/danial-uet/Software-Construction-Labs./tree/master/Lab4

Lab Task 1 — Mutability & Performance (String vs StringBuilder)

Tasks:

1. Objective: Understand the difference between generating temporary copies with immutable types and modifying mutable objects.

2. Implementation (StringPerformance.java):
○ Write a method buildString(int n) that concatenates numbers from 0 to n using a standard String (s = s + i).
○ Write a method buildStringBuilder(int n) that uses StringBuilder.append() to achieve the same result.

3. Test ideas: Measure and print the execution time for n = 10000 to observe the $O(n^2)$ cost of copying immutable strings versus the efficiency of StringBuilder.

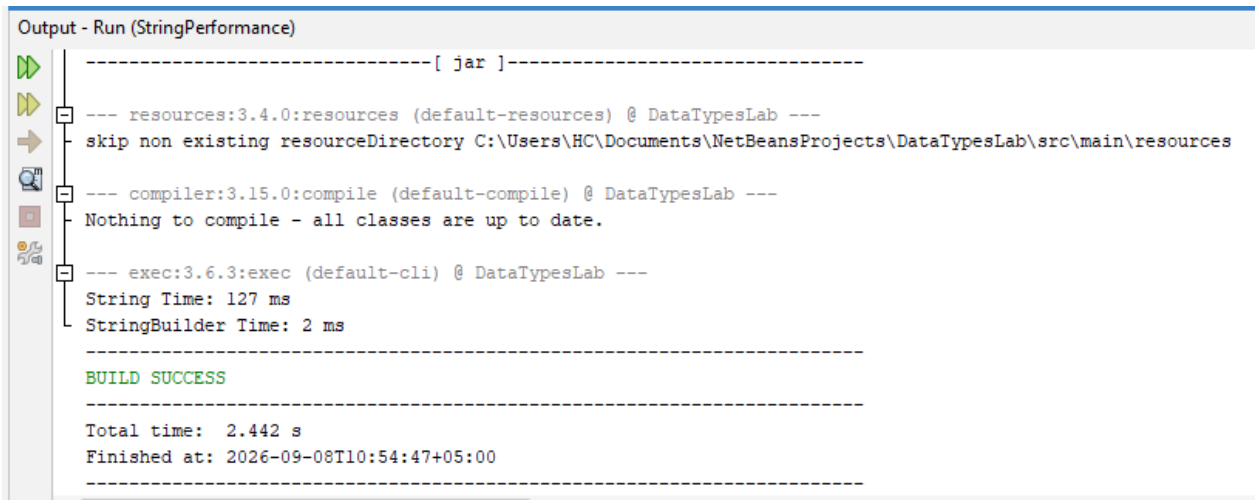
Solution

For first lab task, single Java Class was created by me inside CollectionLab Package in DataTypesLab Project:

● StringPerformance.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab4/src/main/java/CollectionsLab/StringPerformance.java

Output:



3. Reflection Paragraph

In Lab Task 1, I evaluated the performance difference between standard string concatenation and StringBuilder. Standard String concatenation inside a loop creates a new object in memory during each iteration, resulting in an $O(n^2)$ time complexity. In contrast, StringBuilder.append() modifies an internal character array directly, yielding an amortized $O(n)$ time complexity and drastically lower execution time.

Lab Task 2 — Safe Collection Modification

Tasks:

1. Objective: Avoid ConcurrentModificationException by using an Iterator

Implementation (CourseManager.java):

- Create a List subjects containing
  ["6.045", "6.005", "8.03", "6.813", "14.03"] .

  Write a method that attempts to remove items starting with "6." using a for-each loop and observe the runtime exception.
- Fix the method by implementing a while(iter.hasNext()) loop and utilizing iter.remove().

2. Expected Output: The final list should only contain ["8.03", "14.03"]

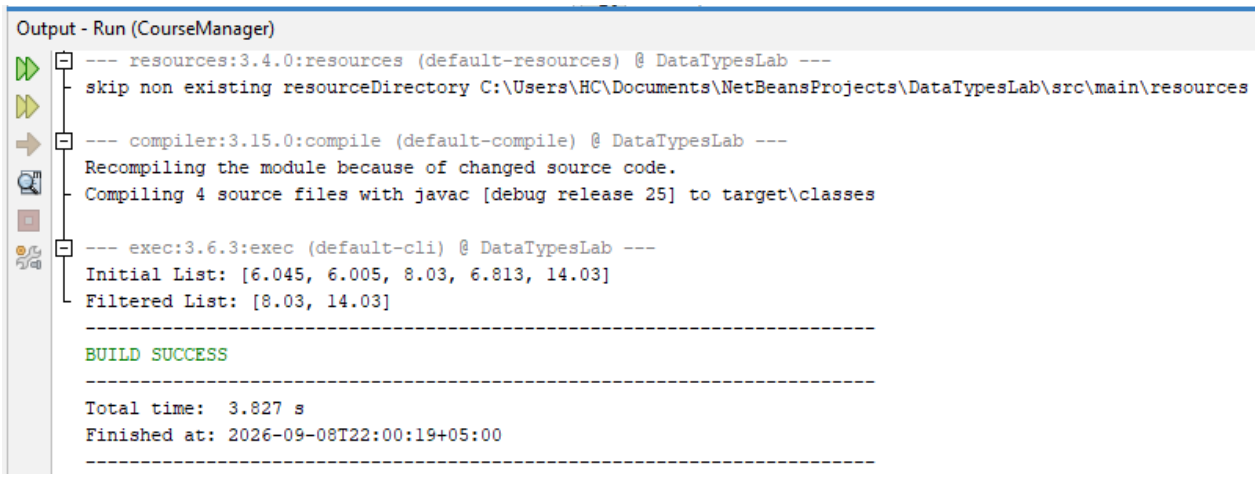
Solution

For second lab task, single Java Class was created by me inside CollectionLab Package in DataTypesLab Project:

● CourseManager.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab4/src/main/java/CollectionsLab/CourseManager.java

Output:



3. Reflection Paragraph

In Lab Task 2, I explored safe collection modification techniques in Java. Modifying an ArrayList using a standard for-each loop while iterating over it triggers a ConcurrentModificationException because the internal modification count changes during iteration. I resolved this issue by using an explicit Iterator and invoking its iter.remove() method, which safely removes elements while updating the internal state of the iterator, ensuring the final list correctly contains only ["8.03", "14.03"].

Lab Task 3 — Complex Data Types

Tasks:

1. Objective: Utilize a ["8.03", "14.03"]. HashMap to store key-value pairs representing a treasure map.

2. Implementation (TreasureMap.java):

```
Map<String, Double> treasures = new HashMap<>();
treasures.put("beach", 25.0);
treasures.put("palm", 50.0);
treasures.put("cove", 75.0);
```

○ Add a new mapping: update the value of "palm" to be its current value plus the size of the map.

○ Iterate through treasures.values() using a total sum of all doubles in the map.

Solution

For third lab task, single Java Class was created by me inside CollectionLab Package in DataTypesLab Project:

● TreasureMap.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab4/src/main/java/CollectionsLab/TreasureMap.java

Output:

```
Output - Run (TreasureMap)
--- resources:3.4.0:resources (default-resources) @ DataTypesLab ---
skip non existing resourceDirectory C:\Users\HC\Documents\NetBeansProjects\DataTypesLab\src\main\resources

--- compiler:3.15.0:compile (default-compile) @ DataTypesLab ---
Recompiling the module because of changed source code.
Compiling 5 source files with javac [debug release 25] to target\classes

--- exec:3.6.3:exec (default-cli) @ DataTypesLab ---
Treasure Map: {chest=100.0, palm=53.0, skull=25.0}
Total Sum: 178.0
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  4.711 s
Finished at: 2026-09-08T22:12:59+05:00
------------------------------------------------------------------------
```

3. Reflection Paragraph

In Lab Task 3, I practiced working with key-value associations using Java's HashMap. I updated existing entries dynamically based on the map's current metadata (size()) and iterated through treasures.values() to accumulate a total sum. A key takeaway from this exercise is that HashMap operations like lookup (get) and insertion (put) run in $O(1)$ average time complexity, while iterating over the collection takes $O(n)$ time.

Lab Task 4 — Unmodifiable Wrappers

Tasks:

1. Objective: Implement immutability at runtime by locking a collection.

2. Implementation (Zoo.java):

- Create a mutable List animals containing "bear" . "lion" ,
- Pass this list into a "tiger" , and Collections.unmodifiableList() wrapper and return it via a getAnimals() method.

3. Test ideas: Attempt to call add("flamingo") on the returned read-only list. Catch and print the UnsupportedOperationException.

Solution

For forth lab task, single Java Class was created by me inside CollectionLab Package in DataTypesLab Project:

● Zoo.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab4/src/main/java/CollectionsLab/Zoo.java

Output:

```
Output - Run (Zoo)
--- resources:3.4.0:resources (default-resources) @ DataTypesLab ---
skip non existing resourceDirectory C:\Users\HC\Documents\NetBeansProjects\DataTypesLab\src\main\resources

--- compiler:3.15.0:compile (default-compile) @ DataTypesLab ---
Recompiling the module because of changed source code.
Compiling 6 source files with javac [debug release 25] to target\classes

--- exec:3.6.3:exec (default-cli) @ DataTypesLab ---
Current Animals: [bear, lion, tiger]
Exception caught: Cannot modify an unmodifiable list!
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.837 s
Finished at: 2026-09-08T22:19:45+05:00
------------------------------------------------------------------------
```

3. Reflection Paragraph

In Lab Task 4, I implemented runtime immutability using Collections.unmodifiableList(). This wrapper pattern prevents accidental modification of encapsulated data by throwing an

UnsupportedOperationException whenever write operations like add() or remove() are attempted on the returned list view. Understanding read-only collection wrappers is essential for designing secure, defensive APIs that protect internal object state from unauthorized external mutation.

Homework 1 — Enhanced Map Tracking

1. Objective: Extend knowledge of Map and Set to track unique student records.

Tasks:

- Implement a class: StudentDirectory that uses a Map where the key is a student ID and the value is the student's name.
- Write a method the map. public Set getAllIDs() that returns the key set of map.
- Ensure that if a user tries to modify the returned Set , an exception is thrown (hint: wrap it using Collections.unmodifiableSet()).
- Provide tests verifying this behavior.

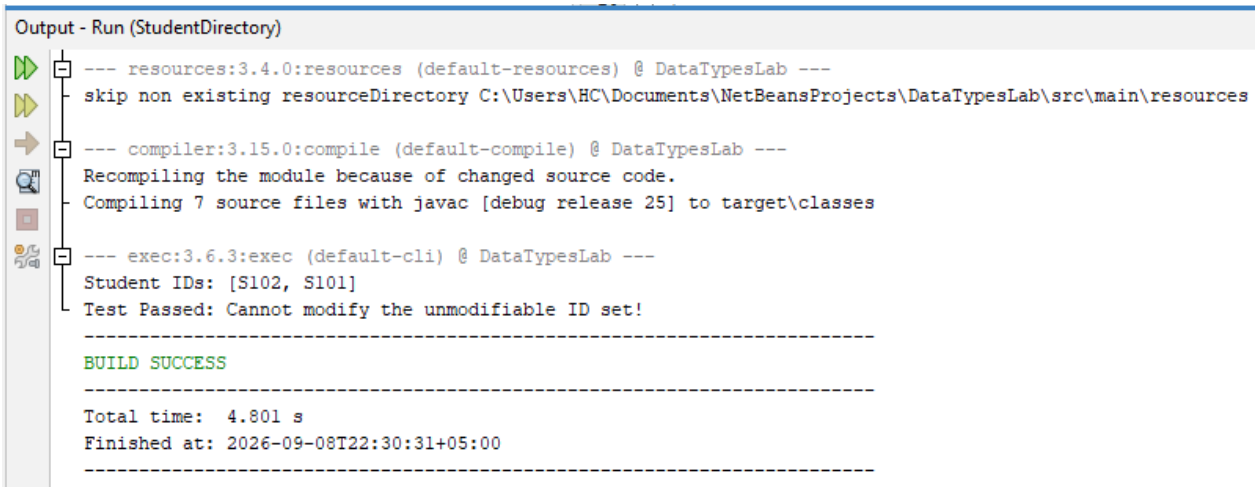
Solution

For first homework, single Java Class was created by me inside HomeWorkTasks Package in DataTypesLab Project:

● StudentDirectory.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab4/src/main/java/HomeWorkTasks/StudentDirectory.java

Output:



3. Reflection Paragraph

In Homework 1, I implemented encapsulation and immutability by wrapping a Set key collection using Collections.unmodifiableSet(). Returning an unmodifiable view directly

protects the internal state of StudentDirectory from unintended external modifications while still granting read-only access to key identifiers. Writing tests to attempt mutations confirmed that write operations like add() fail predictably with an UnsupportedOperationException, maintaining system stability and data integrity.

Homework 2 — Designing Immutable Classes

1. Objective: Apply the final keyword and immutability principles to object-oriented design.

Tasks:

1. Create a fully immutable class called Point.

2. Use the final keyword for both the class declaration and its internal y variables. int x and int y variables.
3. Remove all mutator (setter) methods.

4. Write a reflection in your README explaining why making Point immutable makes it "safer from bugs and easier to understand" as discussed in lecture.

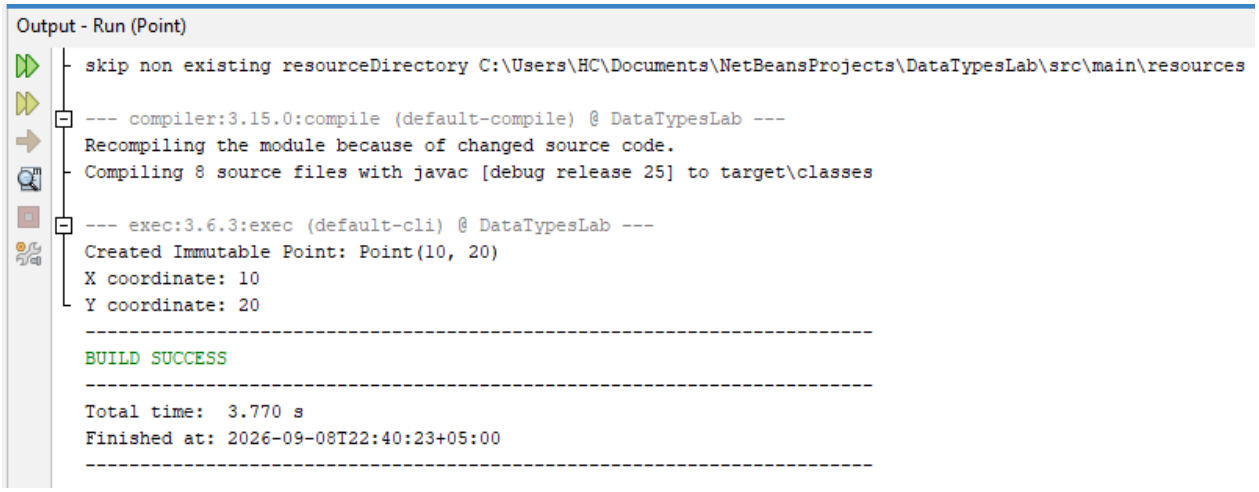
Solution

For first homework, single Java Class was created by me inside HomeWorkTasks Package in DataTypesLab Project:

● Point.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/Lab4/src/main/java/HomeWorkTasks/Point.java

Output:



3. Reflection Paragraph

In Homework 2, I implemented a fully immutable Point class using the final modifier on both the class declaration and its internal x and y primitive fields. Eliminating setter methods guarantees that coordinate values remain fixed throughout the object's lifecycle. Designing classes to be immutable eliminates side effects when passing references across different components of an application, leading to cleaner thread-safety and reliable object behavior.